

CS3.301: Operating Systems and Networks

LECTURE 12&13: Mechanism: Address Translation & Segmentation

INSTRUCTOR: DEEPAK GANGADHARAN

Agenda

■ Part 1: Mechanism — Address Translation

- Base & bounds registers, dynamic relocation
- Hardware support + OS responsibilities

■ Part 2: Segmentation

- Generalizing base/bounds to multiple segments
- Sharing, protection, fragmentation

Recap: Limited Direct Execution

- CPU virtualization used Limited Direct Execution (LDE)
- Let the program run directly on hardware, most of the time
- OS interposes only at key moments (system calls, timer interrupts)
- Goal: efficiency (direct execution) + control (OS interposition)
- **We'll use the SAME strategy to virtualize memory**

Memory Virtualization Problem

- How do we build an **EFFICIENT** virtualization of memory?
- How do we provide the **FLEXIBILITY** applications need?
- How do we maintain **CONTROL** over which memory an app can access?
- ...and do all of this efficiently?
- Answer: hardware-based address translation

Hardware-Based Address Translation

- On EVERY memory reference (fetch, load, store)...
- **...hardware transforms the VIRTUAL address into a PHYSICAL address**
- Hardware alone can't virtualize memory — it's just the low-level mechanism
- The OS must set up the hardware correctly at key points

Simplifying Assumptions (For Now)

- 1. The address space is placed CONTIGUOUSLY in physical memory
- 2. The address space is SMALLER than physical memory
- 3. Every address space is EXACTLY the same size
- **We'll relax all three assumptions as we go along**

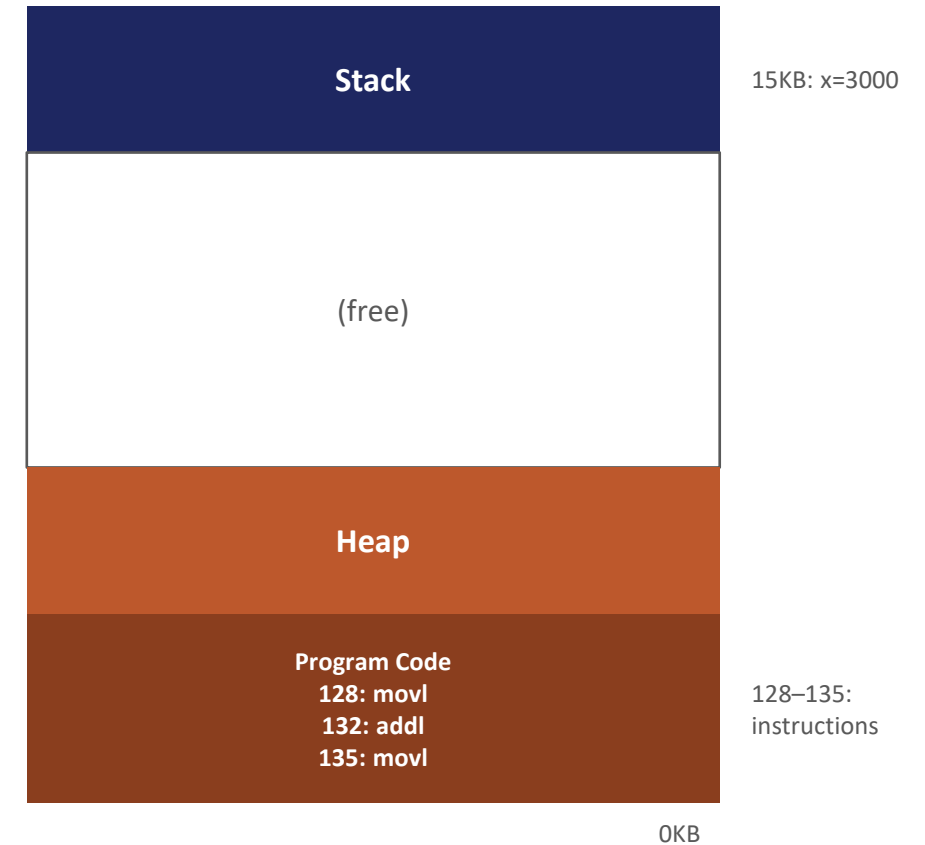
An Example: Process & Address Space

- A process whose address space is 16KB
- Code: `x = x + 3;` compiles to 3 instructions at 128, 132, 135
- Variable `x` lives on the stack at virtual address 15KB

```
void func() {  
    int x = 3000;  
    x = x + 3;  
    ...  
}
```



```
128: movl 0x0(%ebx), %eax    ;load 0+ebx into eax  
132: addl $0x03, %eax       ;add 3 to eax register  
135: movl %eax, 0x0(%ebx)   ;store eax back to mem
```



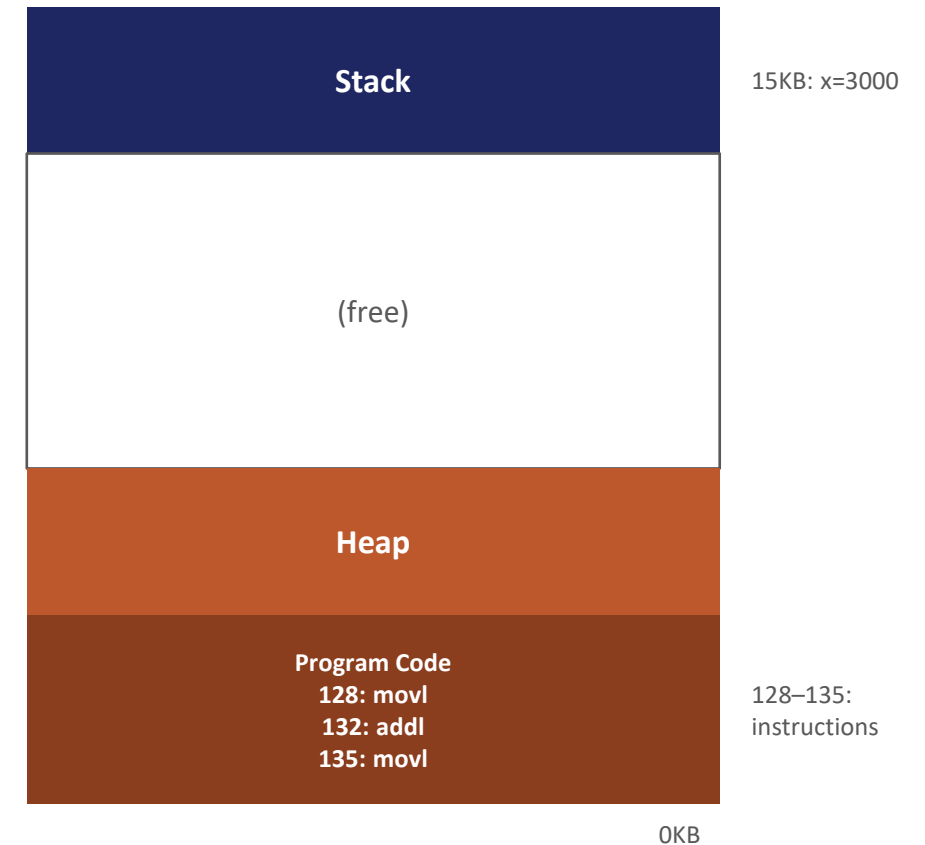
Trace: Memory References During Execution

■ From the process's perspective, running this code generates:

- Fetch instruction at 128
- Execute it — LOAD from address 15KB
- Fetch instruction at 132
- Execute it — no memory reference (just a register op)
- Fetch instruction at 135
- Execute it — STORE to address 15KB

■ **Every one of these is a VIRTUAL address**

```
128: movl 0x0(%ebx), %eax    ;load 0+ebx into eax
132: addl $0x03, %eax        ;add 3 to eax register
135: movl %eax, 0x0(%ebx)    ;store eax back to mem
```

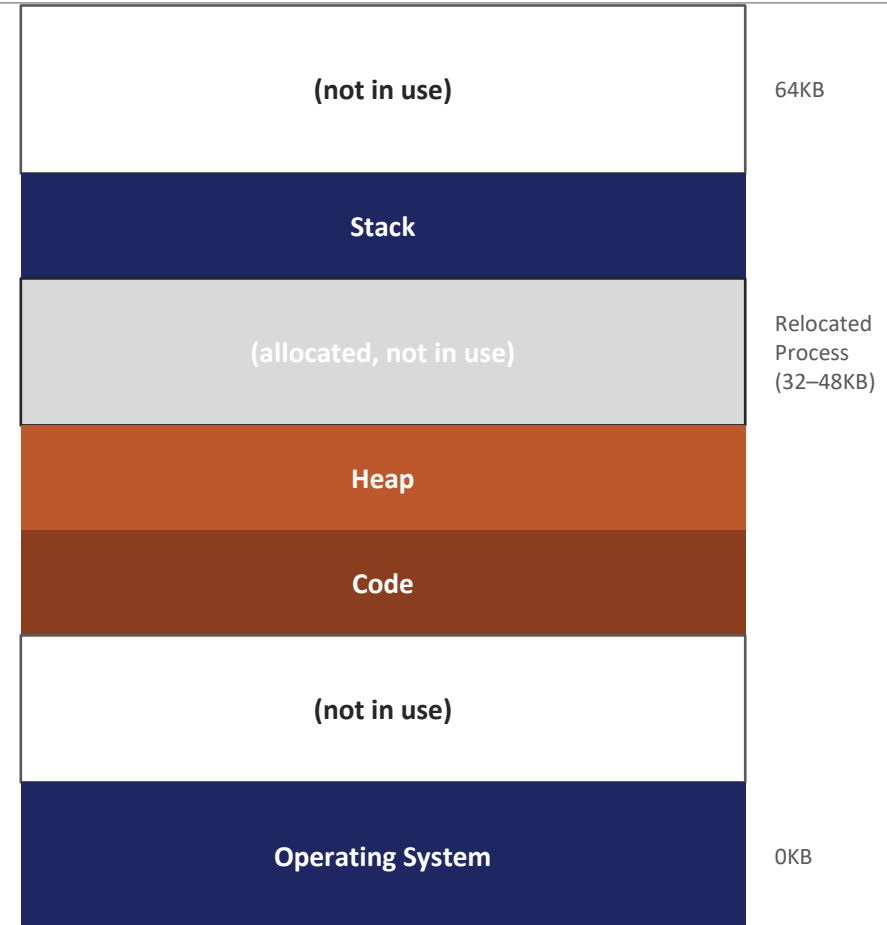


The Relocation Problem

- The process's address space starts at 0 and runs to 16KB
- **But the OS wants to place it somewhere ELSE in physical memory**
- How do we relocate a process transparently?
- How do we preserve the illusion of starting at address 0?

Physical Memory With a Relocated Process

- OS occupies the first slot of a 64KB physical memory
- Our process is relocated to the slot starting at 32KB
- Two other 16KB slots (16–32KB, 48–64KB) remain free



Software-Based Static Relocation

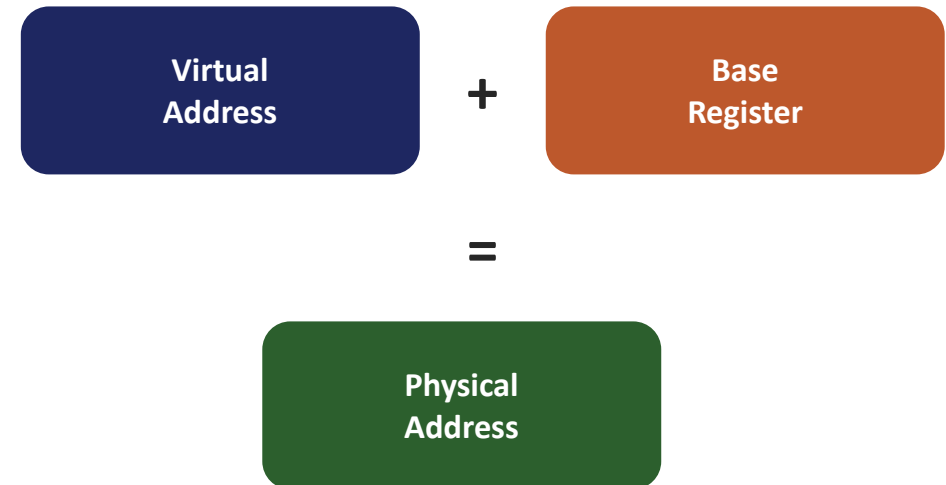
- **Early systems relocated addresses purely in SOFTWARE**
- A 'loader' rewrites every address in the executable at load time
- e.g. load from 1000, loaded at offset 3000 → rewritten to load from 4000
- **Problem #1: NO PROTECTION — a buggy program can still generate bad addresses**
- **Problem #2: hard to relocate again once the process is running**

Dynamic Relocation: Base & Bounds

- Also called the 'base and bounds' approach (late 1950s)
- **Two hardware registers per CPU:**
 - BASE register — where this process is loaded in physical memory
 - BOUNDS (limit) register — the size of the process's address space
- Every program is compiled/written as if loaded at address 0
- The OS sets the base register when the process starts running

The Translation Formula

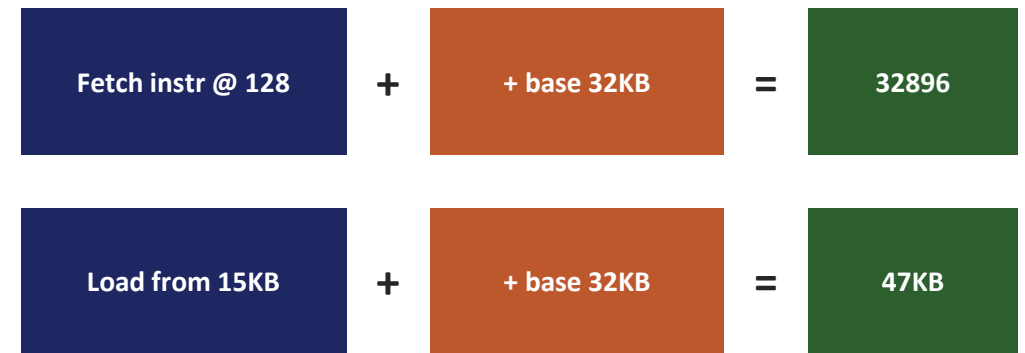
- **Physical address = Virtual address + base**
- Computed by hardware, on every single memory reference



Every memory reference a process makes is a VIRTUAL address; hardware adds the base register automatically, on every access.

Worked Example: Translating Our Instructions

- PC = 128 (fetch instruction)
- 15KB (load/store the variable x)
- Process loaded with base = 32KB (32768 bytes)



The bounds register (16KB) is checked FIRST:
both 128 and 15KB are < 16KB, so both
addresses are legal.

The Bounds Register & Example Translations

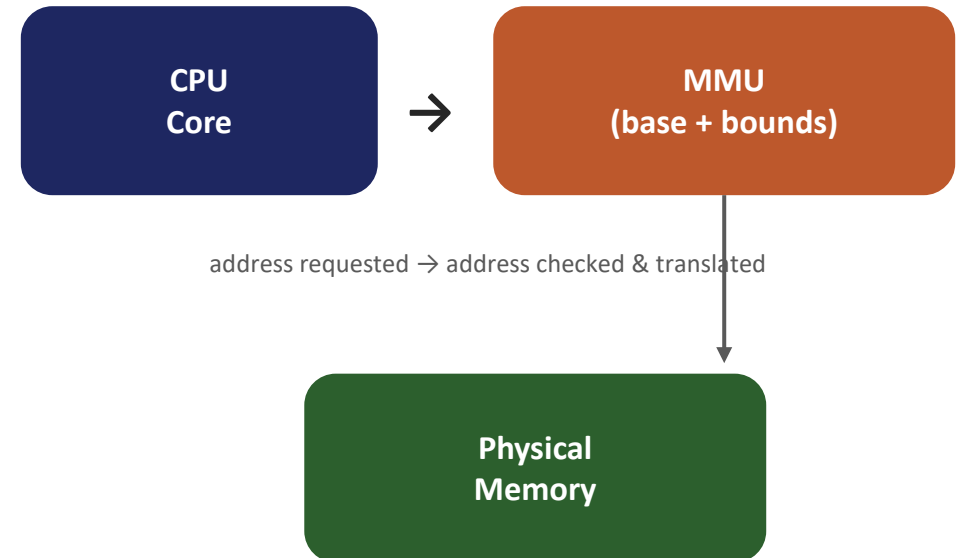
- Bounds register checked FIRST — is the address legal?
- If $VA \geq \text{bounds}$ (or negative) → CPU raises an exception
- Otherwise: $\text{physical} = VA + \text{base}$

Virtual Addr	Physical Addr
0	16 KB
1 KB	17 KB
3000	19384
4400	Fault X

Address space size = 4KB, loaded at physical 16KB

What Is the MMU?

- The Memory Management Unit (MMU) is a piece of hardware built
 - into the processor itself, separate from the main CPU logic
- Its job: transform every address a running program uses
 - into the real address where data actually sits in memory
- Holds the **base/bounds registers** we just introduced
- Grows more sophisticated later in the course (TLBs, page tables)



If the address is out of bounds, the MMU refuses the request and signals the OS instead of letting it reach memory.

Hardware Support: Summary

- Six hardware requirements to support dynamic relocation

Privileged (kernel) mode

Base / bounds registers

Translate + bounds-check circuitry

Privileged instrs to set base/bounds

Privileged instrs to register handlers

Ability to raise exceptions

The Free List

- **The OS must track which physical memory is free vs. in use**
- Simplest data structure: a FREE LIST
- Just a list of the ranges of physical memory not currently in use
- Used whenever a new process needs to be placed in memory

OS Responsibility: Memory Management

- Thanks to our simplifying assumptions, this is straightforward
- Physical memory = array of fixed-size slots

- 1 On process creation: search the free list, find room, mark it used
- 2 On process termination: reclaim all its memory, add back to free list

OS Responsibility: Context Switches

- Every process has different base/bounds values

1

Only ONE base/bounds pair exists per CPU

2

On switch OUT: save base & bounds into the process's PCB

3

On switch IN: restore base & bounds from the new process's PCB

OS Responsibility: Exception Handlers

- The OS must be ready to react to illegal behavior

1

Installed once, at boot time, via privileged instructions

2

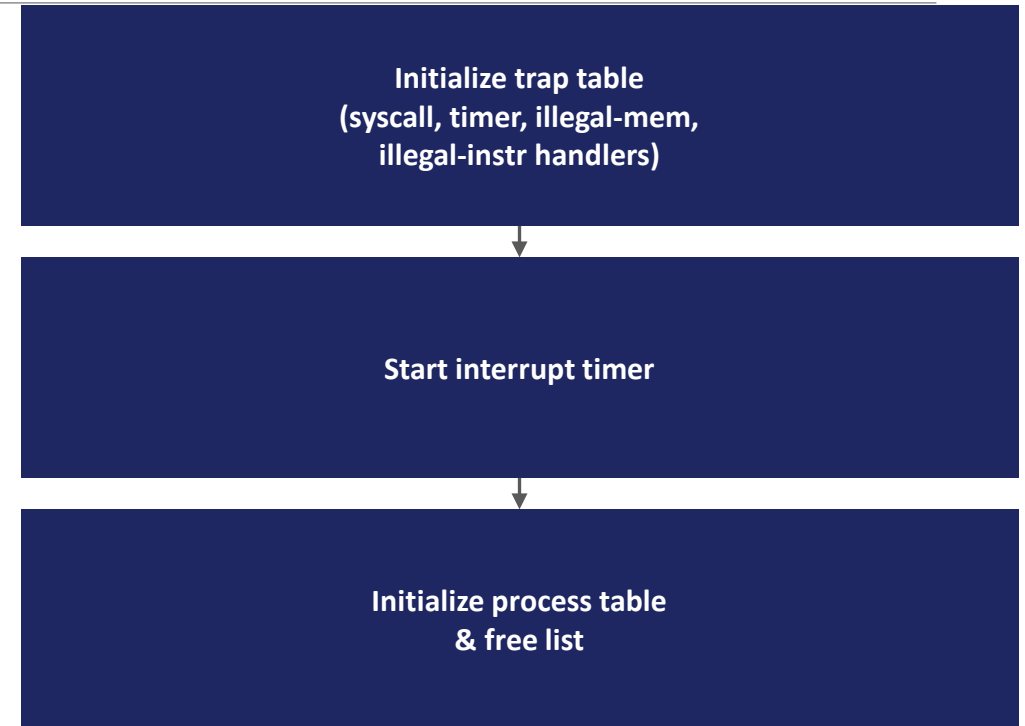
Out-of-bounds access → CPU traps → OS handler runs

3

Common OS reaction: terminate the offending process

Timeline: What Happens at Boot

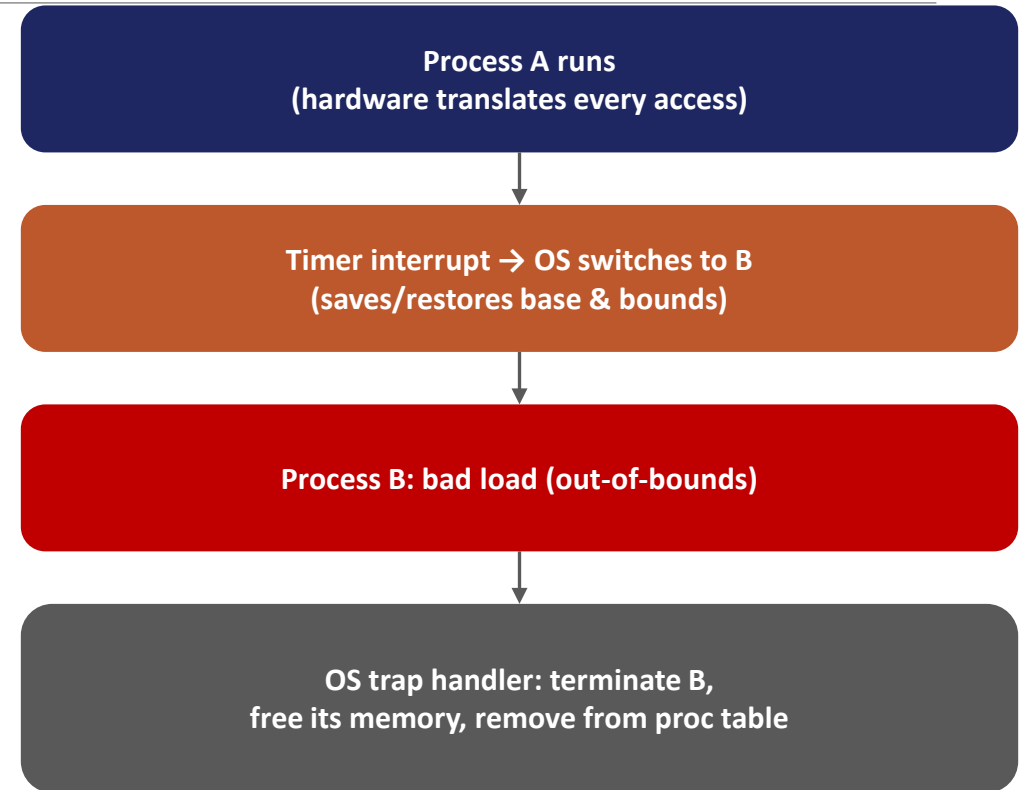
- Before any user process ever runs...



All performed in KERNEL MODE, at boot

Timeline: Runtime — A Runs, B Faults

- Watch how little the OS has to do while things go well

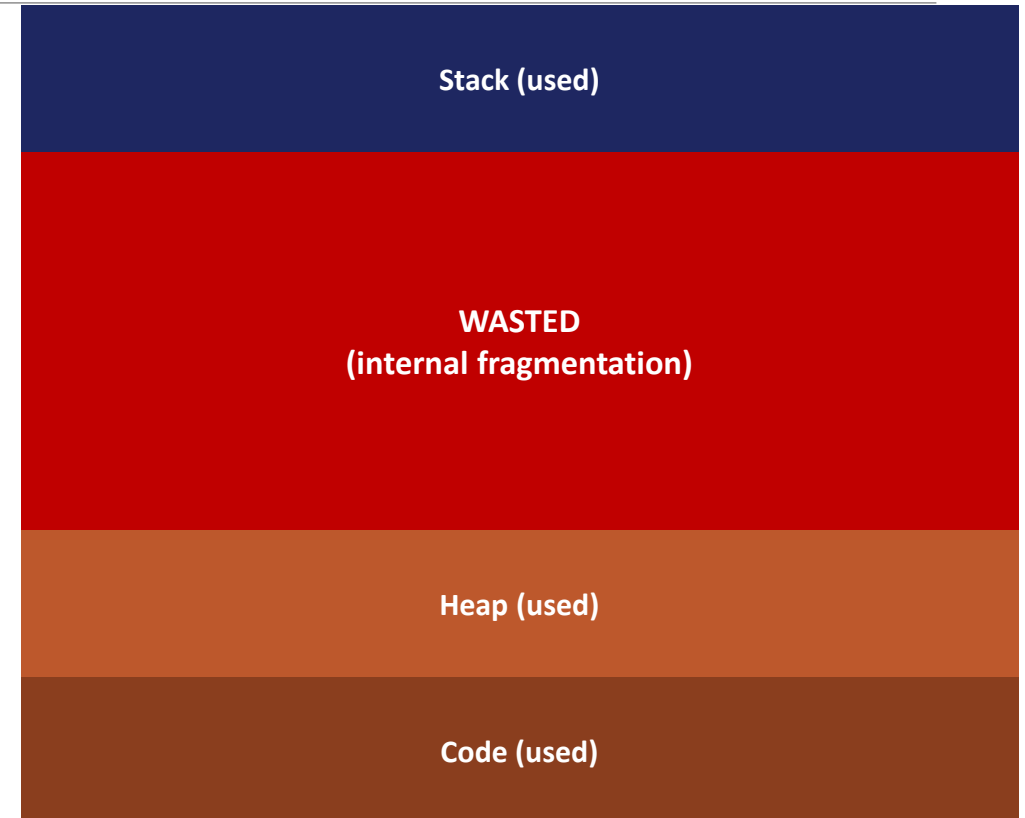


Recap: Dynamic Relocation

- **Base + bounds registers** → simple, fast address translation
- $\text{physical} = \text{virtual} + \text{base}$, checked against bounds
- Provides BOTH efficiency (cheap hardware) and protection
- OS: manages free list, sets base/bounds per process, handles faults

The Cost: Internal Fragmentation

- The process's ENTIRE address space slot must be reserved
- ...even the big FREE gap between stack and heap!
- This wasted space inside an allocated unit = internal fragmentation



Entire 32KB–48KB slot is reserved — even the free middle

Transition: From Base/Bounds to Segmentation

- Base & bounds: one register pair for the WHOLE address space
- Problem: can't avoid reserving the free middle gap
- Next idea: what if we had a base/bounds pair PER SEGMENT?

Motivation: The Wasted Middle

- Look again at a typical address space (code, heap, free, stack)
- The free space between heap and stack is NOT used by the process...
- ...but base & bounds still reserves it in PHYSICAL memory
- Gets far worse for large (e.g. 32-bit, 4GB) address spaces

The Crux: Supporting Large, Sparse Spaces

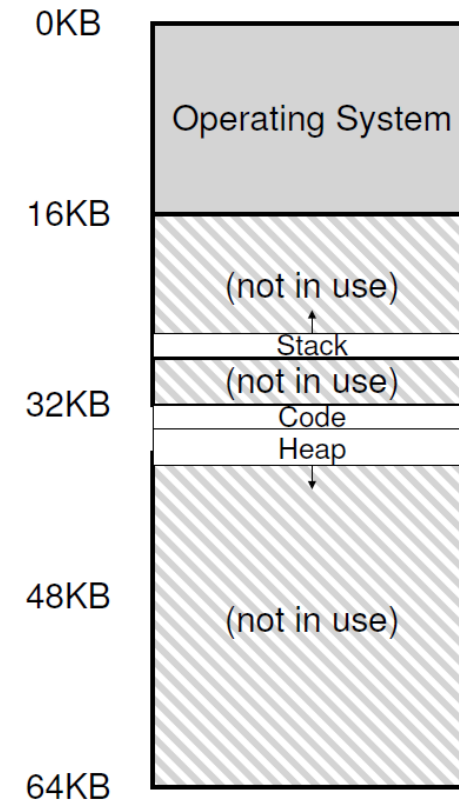
- How do we support a large address space...
- ...with potentially a LOT of free space in the middle?
- We want physical memory to only be used for what's ACTUALLY used
- This property — mostly empty, with used regions scattered around —
- ...is called a **SPARSE address space**

Segmentation: Generalized Base/Bounds

- **Idea (dating to the early 1960s): don't use just ONE base/bounds pair**
- Instead, use a base/bounds pair PER logical SEGMENT
- A segment = a contiguous portion of the address space
- Our canonical 3 segments: CODE, HEAP, STACK
- **Each segment can be placed independently in physical memory**

Placing Segments in Physical Memory

- Each segment (code, heap, stack) gets its own slot
- **Only USED memory occupies physical space**
- Large, sparse address spaces now fit comfortably

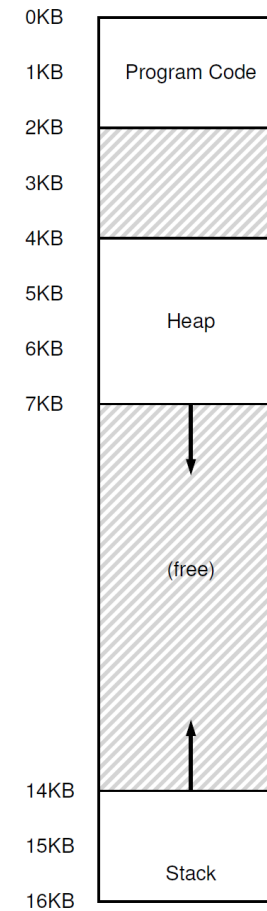


Only USED memory occupies physical space — sparse OK

Segment Register Values

- The MMU now holds 3 base/bounds (size) register pairs
- Size register = same role as the old bounds register

Segment	Base	Size
Code	32K	2K
Heap	34K	3K
Stack	28K	2K



Worked Example: Translating a Code Address

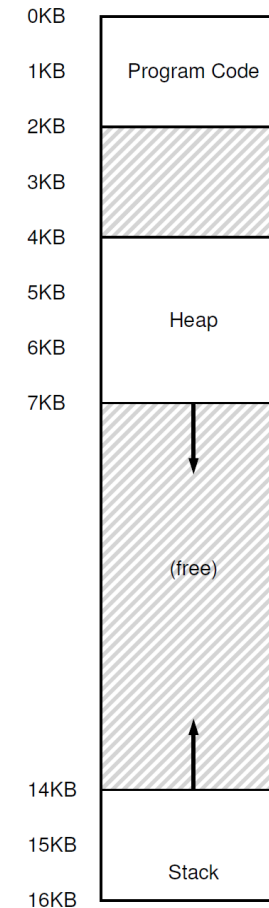
- Virtual address 100 lies in the code segment

Virtual address: 100 (in the CODE segment)

$\text{physical} = \text{offset} + \text{base} = 100 + 32768$

Physical address: 32868

100 < size(code)=2KB, so this is a LEGAL access. Since the code segment starts at virtual address 0, the offset IS simply the virtual address itself.



Worked Example: Translating a Heap Address

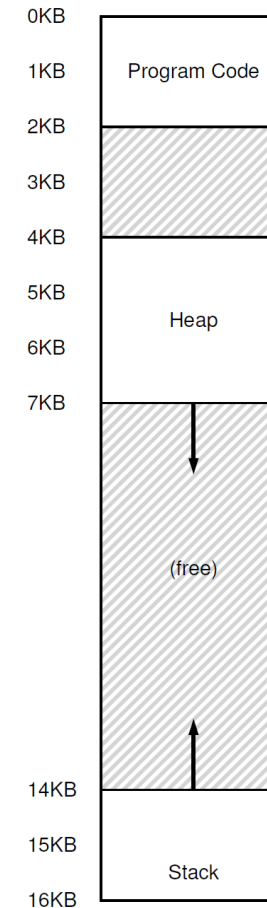
- This one is trickier — the heap does NOT start at virtual 0!

Virtual address: 4200 (in the HEAP segment)

offset = $4200 - 4096$ (heap start) = 104
physical = $104 + 34816$

Physical address: 34920

Naively adding $4200 + 34K$ would give the WRONG answer (39016). We must first compute the OFFSET into the segment, i.e. subtract where the segment begins in the virtual address space, before adding the physical base.



Aside: The Segmentation Fault

- An illegal access on a segmented machine → 'segmentation fault'
- Hardware detects the out-of-bounds access, traps to the OS
- OS likely terminates the offending process
- The term persists today, even on machines with no real segmentation!

Which Segment? The Explicit Approach

- Chop up the virtual address using its TOP few bits
- Our example: 14-bit virtual address, top 2 bits pick the segment



Segment
(top 2
bits)

Offset (bottom 12 bits)

00 = Code 01 = Heap 11 = Stack

SEG_MASK = 0x3000, SEG_SHIFT = 12,
OFFSET_MASK = 0xFFF

```
segment_translation.c

1 // get top 2 bits of 14-bit VA
2 Segment = (VirtualAddress & SEG_MASK) >> SEG_SHIFT
3 // now get offset
4 Offset = VirtualAddress & OFFSET_MASK
5 if (Offset >= Bounds[Segment])
6     RaiseException(PROTECTION_FAULT)
7 else
8     PhysAddr = Base[Segment] + Offset
9     Register = AccessMemory(PhysAddr)
```

Explicit Approach: Full Worked Example

00 = Code 01 = Heap 11 = Stack

- **Virtual address 4200 in binary (14 bits):**

- 01 0000 0110 1000
- Top 2 bits '01' → HEAP segment
- Bottom 12 bits → offset = 0x068 = 104 (decimal)

- **Matches our manual calculation from 2 slides ago!**

- **Issue with using Top 2 bits – Only 3 segments → One segment of the address space goes unused**

- **To fully utilize the address space, some systems put code and heap in the same segment and use only one bit to select the segment**

- **Another issue – Limits the use of virtual address space!**

The Implicit Approach

- **Alternative: infer the segment from HOW the address was formed**
- Address came from the program counter (an instruction fetch)?
 - → must be the CODE segment
- Address came from the stack/base pointer?
 - → must be the STACK segment
- Any other address → must be the HEAP segment

Explicit vs. Implicit: A Side-by-Side Look

- Both answer the same question: which segment is this?
- They just get the answer in different ways

Explicit	Implicit
Reads it straight out of the address's own bits	Infers it from HOW the address was produced
Simple, fast hardware	Saves address bits; slightly more logic
Wastes some bits as a segment selector	No wasted bits, but rules can get tricky

What About the Stack?

- **The stack GROWS BACKWARD — toward lower addresses!**
- Hardware needs one more bit: which direction does this segment grow?

Virtual: 15KB region

grows ↓ toward lower addresses

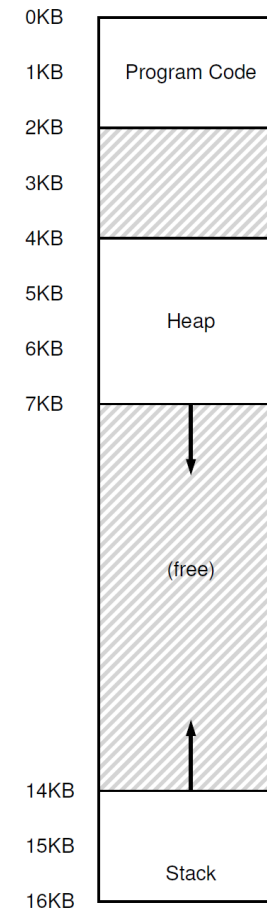
Physical: base 28KB → 27KB

$\text{offset} = 3\text{KB} - \text{max_size}(4\text{KB}) = -1\text{KB}$
 $\text{physical} = \text{base} + \text{negative offset}$
 $= 28\text{KB} + (-1\text{KB}) = 27\text{KB}$

Worked Example: Translating a Stack Address

- Virtual address 15KB (in the stack, which grows backward)
- Max segment size = 4KB; offset = 15KB's position within it
- negative offset = 3KB – 4KB (max size) = –1KB**
- physical = base(28KB) + (–1KB) = 27KB**
- Bounds check: $|-1KB| \leq \text{current stack size (2KB)}$? Yes — legal

Segment	Base	Size
Code	32K	2K
Heap	34K	3K
Stack	28K	2K



Support for Sharing: Protection Bits

- **Idea: let multiple processes SHARE certain segments**
- Most common case: sharing CODE (e.g. shared libraries)
- Needs extra hardware: protection bits per segment
- Read / Write / Execute permissions, checked on every access
- **Code segment: Read-Execute only → safe to share, can't be modified**

Fine-Grained vs. Coarse-Grained Segmentation

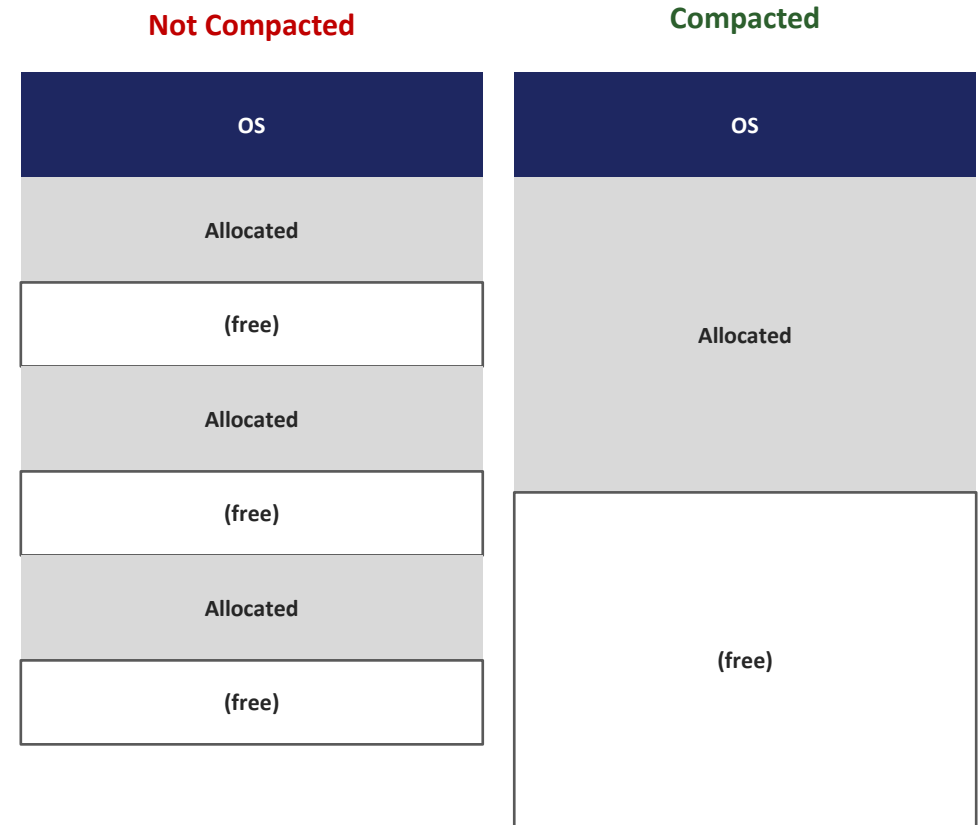
- **COARSE-GRAINED:** just a few big segments (code, heap, stack)
- What we've discussed so far — simple, limited flexibility
- **FINE-GRAINED:** many, many small segments
- Requires a SEGMENT TABLE stored in memory (not just a few registers)
- e.g. Multics, Burroughs B5000 — thousands of segments possible
- Lets the OS manage memory usage far more precisely

OS Support for Segmentation

- **Context switches: save/restore ALL segment registers**
- Growing a segment: e.g. `malloc()` triggers heap growth
 - Library issues a system call (e.g. `sbrk()`) to grow the heap segment
 - OS updates the segment's size register, if space allows
- **Managing free space in physical memory — now much harder!**

The Cost: External Fragmentation

- Variable-sized segments → physical memory quickly becomes full of little holes of free space
- 24KB free total, but scattered in 3 separate chunks
- A 20KB request FAILS — even though 24KB is technically free!



Fighting Fragmentation: Compaction & Allocators

- **Option 1: COMPACTION** — rearrange segments to consolidate free space

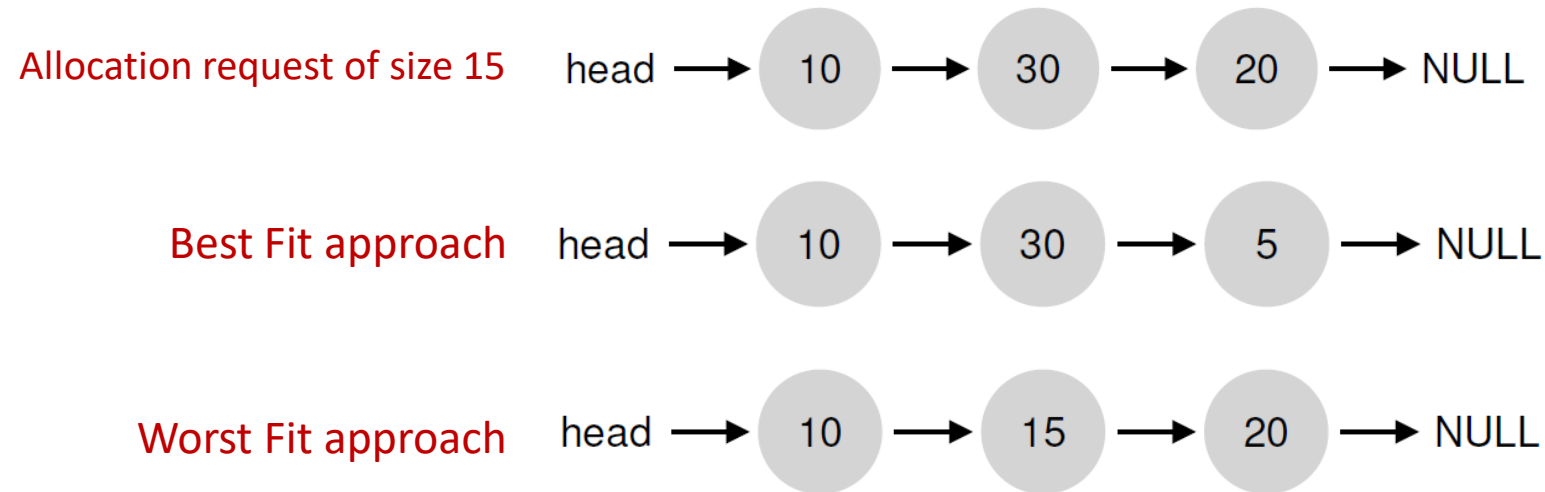
- Stop processes, copy their segments together, update registers
- Works, but expensive — lots of copying, uses real CPU time

- **Option 2: smarter FREE-LIST algorithms**

- best-fit, worst-fit, first-fit, buddy algorithm...

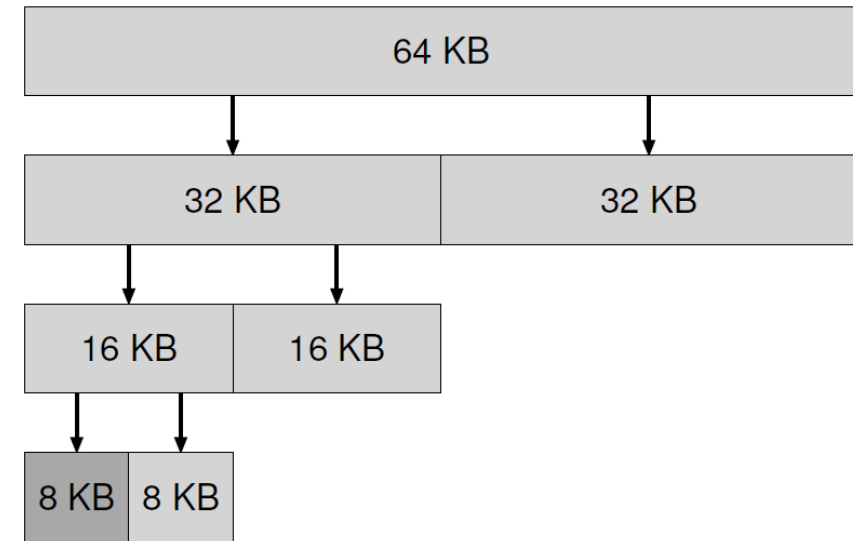
- **Hundreds of algorithms exist — none fully eliminates the problem**

Few Strategies with Free List



Buddy Allocation

- 64 KB free space getting divided in search for a 7 KB block
- When a block is freed, the allocator checks if the buddy block is free → If free, it coalesces the two blocks



Two Kinds of Waste: A Recap

- Both mechanisms we've covered waste physical memory
- ...but for very different reasons

Internal	External
Waste INSIDE a region that was handed out	Waste BETWEEN regions that were handed out
Seen with: one base/ bounds pair per process	Seen with: variable-sized segments
Fix: give each part its own region (segmentation)	Fix: compaction, or smarter free-list picking

Summary: Segmentation

- **WINS: supports large, sparse address spaces efficiently**
- Fast translation (simple hardware arithmetic)
- Bonus: enables safe memory SHARING (e.g. shared code)
- **REMAINING PROBLEMS:**
- External fragmentation — variable-sized segments leave holes
- Doesn't help if ONE segment (e.g. heap) is itself huge & sparse
- **Next up: a fix that avoids variable-sized allocation entirely — PAGING**

Thank You!